AURA Technical Learning Manual

A From-Scratch Guide for Understanding, Rebuilding, Testing, and Explaining the P

Project: AURA ? Autonomous University Response and Action
Department: Computer Science & Engineering, TKM College of Engineering
Current implementation position: Approximately 70% complete
Purpose: Learning and technical understanding, not a presentation script
Source of truth: The Java source, SQL scripts, tests, repository documentation, and the first presentation ownership matrix

How To Use This Manual

This file is written for a team that has a working project but wants to understand how it was built. It should be read as a small project-building course.

Read the manual in this order:

1. Understand the problem AURA solves.
2. Learn the project technologies and the normal development process.
3. Learn the architecture and the responsibility of each layer.
4. Follow the database and model design.
5. Trace the complete workflows from UI to database.
6. Open the referenced source files and compare them with the explanations.
7. Study your assigned member section in detail.
8. Run the tests and reproduce the golden-path workflows.
9. Compare the implemented work with the remaining work before finalization.

This manual does not claim that planned classes exist when they are not present in the current source tree. The repository contains approved design documents that mention future or planned services. Those are clearly marked as planned/documented. The code descriptions in this manual are based on classes that actually exist in this branch.

1. What AURA Is

1.1 The problem

Students may notice a broken projector, network problem, electrical issue, maintenance problem, or campus improvement opportunity. Without a structured system, the report can be lost in informal messages. Administrators also need a queue, priorities, status history, and resolution notes.

AURA provides:

- Student authentication.
- Anonymous campus issue and suggestion submission.
- Private student tracking of submitted tickets.
- Crowd support through a hype vote.
- An administrator queue.
- Status transitions from PENDING through resolution or rejection.
- Official resolution notes.
- An audit history of status changes.
- Database-level rules for security and data integrity.

1.2 The privacy idea

AURA separates the public submission from the private ownership record.

The submissions table does not contain student_id. This means an administrator reading the submission queue cannot directly identify the student who created a ticket.

The private relationship is held in student_submission_receipts:

```
users
  |
  | private student_id
  v
student_submission_receipts
  |
  | submission_id
  v
submissions
```

The student can find their own submission through the receipt table, while the administrator works with anonymous submission data.

1.3 Current implementation position

The working core is approximately 70% complete. The current branch contains working or substantially implemented code for:

- Java 17 project setup.
- Maven dependency management.
- Swing and FlatLaf desktop entry point.
- User persistence.
- Password authentication.
- Google OAuth flow structure.

- Domain validation for @tkmce.ac.in.
- Session storage and role checks.
- Student submission creation.
- Private receipt creation and lookup.
- Hype persistence and counting.
- Admin submission listing.
- Status updates.
- Resolution notes.
- Submission history.
- PostgreSQL schema and seed data.
- Unit and regression tests.

The main remaining work is around:

- Report generation.
- Analytics and richer dashboard metrics.
- Notifications.
- UI polishing.
- Wider integration and DAO tests.
- More complete history and state edge cases.
- Bug fixing and documentation closure.

2. Technology Used in This Project

2.1 Java 17

Java is the main programming language. The application logic, desktop UI, service layer, database access, authentication, models, and utilities are written in Java.

Java concepts visible in the project include:

- Classes and objects.
- Interfaces.
- Encapsulation through private fields and public methods.
- Constructors and dependency injection through constructor parameters.
- Inheritance concepts in the approved design.
- Collections such as List and ArrayList.
- Exceptions and checked SQLException handling.
- File/package organization.
- Event-driven GUI programming.
- Threads through SwingWorker for browser authentication.
- Records through GoogleOAuthService.GoogleUserInfo.
- Enumerated or state values represented by strings in the current implementation.

The current source branch does not contain the aura.enums package shown in some architecture documents. The current model and DAO methods use strings such as STUDENT, ADMIN, ISSUE, SUGGESTION, PENDING, and RESOLVED.

2.2 Maven

Maven builds the Java project and downloads dependencies from pom.xml.

The important dependencies are:

Dependency	Purpose
PostgreSQL JDBC driver	Connect Java to PostgreSQL/Supabase
FlatLaf	Modern Swing look and feel
jBCrypt	Password hashing and verification
Gson	Parse Google OAuth JSON responses
JUnit Jupiter	Automated tests

Useful commands:

```
# Run all tests
mvn test

# Build without running tests
mvn -DskipTests package

# Run the application through the configured Maven runner
mvn exec:java -Dexec.mainClass=aura.Main
```

2.3 Swing and FlatLaf

The application UI is a desktop UI, not a web UI. It uses Java Swing components such as:

- JFrame for windows.
- JPanel for groups of controls.
- JButton for actions.
- JTextField and JPasswordField for input.
- JTable for submission lists.
- JDialog for the new-submission form.
- JOptionPane for messages and simple prompts.

Main.java applies FlatDarkLaf before opening the login window.

The repository also contains mvp_dashboard.html, but that is a visual reference/MVP file. It is not the main runtime UI. The real application screens are Java Swing classes under src/main/java/aura/ui.

2.4 JDBC and PostgreSQL/Supabase

JDBC is the Java API used to communicate with a relational database.

The flow is:

```
Java UI
  -> service class
    -> DAO class
      -> DatabaseConfig
        -> JDBC DriverManager
          -> PostgreSQL/Supabase
```

A DAO uses Connection, PreparedStatement, and ResultSet. The DAO is responsible for SQL. The UI must not execute SQL directly.

2.5 BCrypt

PasswordUtil uses BCrypt to hash passwords before storing them and to verify a supplied password during authentication.

The database stores a hash, not the original password.

2.6 Google OAuth and Gson

GoogleOAuthService implements the structure of a desktop Google OAuth authorization-code flow:

1. Create OAuth state and PKCE values.
2. Start a local callback server.
3. Open the browser.
4. Receive the callback.
5. Check the state value.
6. Exchange the authorization code for a token.
7. Request user profile information.
8. Return a GoogleUserInfo record to AuthService.

Gson parses JSON responses from Google.

2.7 JUnit 5

JUnit tests verify rules without requiring a person to click every button manually.
The current test suite contains 12 passing tests across six test classes.

3. How To Build A Project Normally Without An Agent

An agent can type code quickly, but it should not replace project understanding. A normal engineering process is:

Step 1: Write the problem

Before writing Java, state the user problem in plain language.

For AURA:

> Students need a trusted channel to report campus issues or suggestions, and administrators need an organized, auditable way to resolve them without exposing student identity.

Step 2: Identify actors and actions

The two main actors are:

- STUDENT: logs in, submits, views own submissions, reads updates, and adds hype.
- ADMIN: logs in, views the queue, changes status, adds resolution notes, and reviews history.

Step 3: Write requirements

The repository records requirements in docs/PRD.md and docs/TRD.md. Requirements become the reason for each later class.

Examples:

Requirement	Later implementation
Student must log in	LoginFrame, AuthService, UserDAO, SessionContext
Only TKMCE users are allowed	ValidationUtil, database email check
Students submit issues and suggestions	SubmissionFormPanel, Submission, SubmissionDAO
Students view only their own tickets	StudentReceiptDAO and receipt table
Admin changes state	AdminDashboardFrame, DatabaseSubmissionService, SubmissionDAO
State changes are auditable	SubmissionHistory, submission_history, transaction
One student cannot hype twice	SubmissionHypeDAO, unique database constraint

Step 4: Draw the architecture

Decide which layer owns each responsibility before coding:

```
aura.ui       -> user actions and display
aura.service  -> business rules and workflow coordination
aura.dao      -> SQL and JDBC
aura.model    -> data objects
aura.config   -> external configuration
aura.util     -> shared rules and session/password helpers
PostgreSQL    -> durable data and constraints
```

Step 5: Design the database

Write sql/schema.sql before writing complicated DAO methods. The schema gives the application a stable contract: table names, columns, keys, checks, indexes, and relationships.

Step 6: Create models

Create simple Java classes for data that moves between layers. A model should not open a database connection.

Step 7: Build one vertical slice

Do not build every UI class first. Build one complete path:

```
Login button
  -> AuthService
    -> UserDAO
      -> users table
        -> User
          -> SessionContext
            -> dashboard
```

Then test it. Repeat for submission and admin resolution.

Step 8: Add tests while building

For every rule, write a test or a repeatable verification procedure. Test invalid input and failure cases, not only successful cases.

Step 9: Integrate the team work

Members must agree on method names, field names, table names, IDs, status values, and error behavior. Compilation alone does not prove that two modules integrate correctly.

Step 10: Finish honestly

Compare the requirements to the actual files. Mark a feature completed only when the code, test, and user-visible behavior exist.

4. Repository Structure

```
AURA/
|-- pom.xml
|-- README.md
|-- AGENT.md
|-- mvp_dashboard.html
|-- docs/
|-- sql/
|-- src/main/java/aura/
|-- src/main/resources/
|-- src/test/java/aura/
|-- src/test/resources/
‘-- tools/
```

4.1 docs/

- ARCHITECTURE.md: layer rules, diagrams, and ownership matrix.
- AUTH_TASK_GUIDE.md: Google OAuth setup and authentication design.
- DATABASE.md: ER model, table descriptions, indexes, and Supabase configuration.
- PRD.md: product requirements and user actions.
- ROADMAP.md: planned phases and progress evidence.
- SECURITY.md: domain checks, BCrypt, SQL injection rules, RBAC, and anonymity.
- TESTING.md: test strategy and regression objectives.
- TRD.md: technology choices, requirements, and curriculum concepts.
- AURA.pdf: first presentation and approved ownership/design material.

4.2 sql/

- schema.sql: creates tables, constraints, and indexes.
- seed.sql: inserts demonstration users, tickets, votes, notes, and history.

4.3 src/main/java/aura/

- Main.java: application startup.
- config/: database and OAuth configuration.
- dao/: SQL/JDBC data access.
- model/: Java data objects.
- service/: business workflows.
- ui/: Swing screens.
- util/: validation, password, and session helpers.

4.4 src/test/java/aura/

Contains authentication, submission-service, hype, utility, and anonymity regression tests.

5. Architecture In Detail

5.1 Presentation layer

The UI layer contains the windows and controls the user interacts with:

- LoginFrame
- StudentDashboardFrame
- SubmissionFormPanel
- AdminDashboardFrame

These classes respond to button clicks and show data. They call service classes rather than writing SQL.

5.2 Service layer

The service layer coordinates actions and applies business rules:

- AuthService: login, registration, OAuth, logout, role checks.
- GoogleOAuthService: browser OAuth protocol.
- DatabaseSubmissionService: concrete submission-service implementation.
- HypeService: hype behavior.
- ServiceRegistry: supplies the submission service.
- SubmissionService: interface defining submission operations.

The service layer is where an action becomes a business operation. For example, ?change status? is not merely one SQL update: the service must know the old status, request the new status, identify the admin, and ensure history is recorded.

5.3 DAO layer

The DAO layer is the database boundary:

- UserDAO: user CRUD and lookup.
- SubmissionDAO: submissions, receipts, status history, and resolution notes.
- StudentReceiptDAO: private student receipt lookup.
- SubmissionHypeDAO: hype insert, remove, count, and duplicate checks.

5.4 Model layer

Models carry data:

- User
- Submission
- ResolutionNote
- SubmissionHistory

They contain fields and getters/setters. They do not know how to connect to PostgreSQL.

5.5 Configuration and utilities

DatabaseConfig creates connections from local properties or environment values. OAuthConfig reads OAuth settings. PasswordUtil, ValidationUtil, and SessionContext provide shared rules.

5.6 Dependency direction

The intended direction is downward:

```
LoginFrame
  -> AuthService
    -> UserDAO
      -> DatabaseConfig
        -> PostgreSQL
```

A class should not skip its layer. For example, LoginFrame should not call UserDAO directly, and AuthService should not construct a Connection itself.

6. Database Design From Scratch

6.1 Tables

The actual schema creates six main tables.

users

Stores students and administrators.

Important columns:

- user_id: primary key.
- name.
- email: unique and restricted to @tkmce.ac.in.
- password_hash.
- role: STUDENT or ADMIN.
- created_at.

submissions

Stores anonymous issues and suggestions.

Important columns:

- submission_id: primary key.
- title.
- description.
- type: ISSUE or SUGGESTION.
- category.
- location.
- priority.
- status.
- photo_url.
- created_at.

There is intentionally no student_id column.

student_submission_receipts

Stores the private mapping between a student and a submission.

- receipt_id: primary key.
- student_id: foreign key to users.
- submission_id: foreign key to submissions.
- Unique pair: one student cannot receive the same submission twice.

submission_hype

Stores student support votes.

- hype_id: primary key.
- submission_id: foreign key.
- student_id: foreign key.
- Unique pair: one student cannot hype the same submission twice.

resolution_notes

Stores official administrator notes.

- note_id.
- submission_id.
- admin_id.
- note.
- created_at.

submission_history

Stores status transitions.

- history_id.
- submission_id.
- old_status.
- new_status.
- changed_by.
- changed_at.

6.2 Constraints

The database is not just passive storage. It enforces important rules:

- Primary keys identify rows.
- Foreign keys prevent references to missing users or submissions.
- Email checks prevent non-institutional addresses.
- Status checks restrict values to the lifecycle states.
- Unique constraints prevent duplicate receipts and hypes.
- Indexes improve lookup by status, category, priority, hype, and student receipt.

6.3 Setup order

1. Create or open the PostgreSQL/Supabase project.
2. Run sql/schema.sql.
3. Run sql/seed.sql.
4. Copy src/main/resources/db.properties.example to a local db.properties.
5. Set the database URL, user, password, and driver.
6. Run tests or start the application.

6.4 Seed data

The seed script contains:

- Four student users.
- Two admin users.
- Five sample submissions.
- Private receipt mappings.
- Hype votes.
- Resolution notes.
- Status history rows.

The seed data exists so the team can learn and demonstrate the application without first entering every record manually.

7. Class-by-Class Learning Map

7.1 Main.java

Location: src/main/java/aura/Main.java

Purpose:

1. Apply FlatDarkLaf.
2. Start Swing work through SwingUtilities.invokeLater.
3. Create and display LoginFrame.

Study question: Why should the UI start on the Swing event thread? Because Swing UI work is expected to be coordinated through its event-dispatching mechanism.

7.2 DatabaseConfig.java

Location: src/main/java/aura/config/DatabaseConfig.java

Purpose:

1. Load db.properties from the classpath.
2. Read db.url, db.user, and db.password.
3. Allow environment-variable fallback.
4. Load the PostgreSQL driver.
5. Return a JDBC Connection.

It prevents every DAO from duplicating connection setup.

7.3 OAuthConfig.java

Loads OAuth values such as client ID, client secret, redirect URI, server port, enforced domain, and development mode.

7.4 User.java

Represents a user. Current fields include the user ID, name, email, password hash, role, and creation data as defined by the current model.

The user object is used by UserDAO, AuthService, SessionContext, and the UI routing logic.

7.5 Submission.java

Represents a public anonymous ticket. It contains issue/suggestion data and status information but no student-author field.

It is used by the submission service, DAO, student dashboard, and admin dashboard.

7.6 ResolutionNote.java

Represents an official note written by an administrator for a submission.

7.7 SubmissionHistory.java

Represents a status transition and the administrator/user ID that caused it.

7.8 AuthService.java

Main authentication coordinator.

Important methods:

- authenticate(String email, String password).
- registerStudent(String name, String email, String password).
- loginWithGoogle().
- loginMock(String role).
- logout(int userId) and logout().
- isCurrentUserAdmin().
- isCurrentUser(User user).
- validateCredentials(...).

The service validates input, calls the user DAO, verifies passwords, provisions OAuth students, and updates the session.

7.9 GoogleOAuthService.java

Implements the browser-based OAuth flow. It uses:

- HttpServer for the local callback.
- Desktop to open the browser.
- HttpClient for token/profile requests.
- PKCE verifier/challenge values.
- A cryptographic state value.
- Gson JSON parsing.

7.10 DatabaseSubmissionService.java

Implements SubmissionService and coordinates submission operations through SubmissionDAO.

It connects UI requests to DAO behavior and turns low-level data failures into service-level runtime failures for the UI.

7.11 HypeService.java

Provides the business-facing hype operations:

- Add hype.
- Remove hype.
- Count hype.
- Check whether a student already hyped a ticket.

It delegates persistence to SubmissionHypeDAO.

7.12 ServiceRegistry.java

Provides the submission service used by both dashboards. It is a small composition point so the UI does not construct all database dependencies itself.

7.13 DAOs

UserDAO

Performs user insert, lookup, update, and delete operations.

SubmissionDAO

Performs submission persistence, full listing, ID lookup, receipt creation, status changes, history, and resolution-note operations.

StudentReceiptDAO

Performs private ownership mapping operations.

SubmissionHypeDAO

Performs hype persistence and checks.

7.14 UI classes

LoginFrame

Collects password or OAuth login and routes the user to the correct dashboard.

StudentDashboardFrame

Shows student submissions and provides new submission, refresh, hype, detail, and logout actions.

SubmissionFormPanel

Collects title, description, type, category, location, priority, and photo URL information.

AdminDashboardFrame

Shows the administrator queue and provides refresh, status change, note, history, and logout actions.

7.15 Utilities

PasswordUtil

Hashes and verifies passwords with BCrypt.

ValidationUtil

Validates institutional email addresses and minimum password length and normalizes email values.

SessionContext

Stores the current authenticated User and exposes role checks.

8. Code Patterns Used in the Project

8.1 Model object creation

The UI creates a Submission object from form fields. It does not write SQL:

```
Submission submission = new Submission();
submission.setTitle(title);
submission.setDescription(description);
submission.setType(type);
submission.setCategory(category);
submission.setLocation(location);
submission.setPriority(priority);
submission.setPhotoUrl(photoUrl);
```

The object then crosses into the service layer.

8.2 Service call

The student dashboard uses the service contract:

```
SubmissionService service = ServiceRegistry.getSubmissionService();
service.createSubmission(submission, user.getUserId());
```

The UI knows the operation it needs, not the SQL details.

8.3 Prepared statement

A DAO uses parameters rather than string concatenation:

```
String sql = "SELECT user_id, name, email, password_hash, role "
        + "FROM users WHERE email = ?";

try (Connection connection = DatabaseConfig.getConnection();
     PreparedStatement statement = connection.prepareStatement(sql)) {
    statement.setString(1, ValidationUtil.normalizeEmail(email));
    try (ResultSet results = statement.executeQuery()) {
        return results.next() ? mapUser(results) : null;
    }
}
```

This is safer and keeps database values separate from SQL syntax.

8.4 Row mapping

A DAO reads a ResultSet and constructs a model:

```
User user = new User();
user.setUserId(results.getInt("user_id"));
user.setName(results.getString("name"));
user.setEmail(results.getString("email"));
user.setPasswordHash(results.getString("password_hash"));
user.setRole(results.getString("role"));
return user;
```

8.5 Transaction

Creating a submission and its receipt must succeed together:

```
connection.setAutoCommit(false);
// insert submission
// read generated submission ID
// insert private receipt
connection.commit();
```

If a database exception occurs, the code rolls back. This prevents a submission without a receipt or a receipt pointing to a missing submission.

8.6 Event-driven UI

A button receives an action listener. The listener calls a service and refreshes the UI:

```
newSubmission.addActionListener(event -> openSubmissionDialog());
refresh.addActionListener(event -> loadSubmissions());
```

The application waits for the user event instead of running in a fixed command-line sequence.

8.7 Background OAuth work

Google login uses SwingWorker so a browser/network operation does not freeze the Swing UI:

```
new SwingWorker<User, Void>() {
    @Override
    protected User doInBackground() throws Exception {
        return authService.loginWithGoogle();
    }

    @Override
    protected void done() {
        // Read the result and open the dashboard.
    }
}.execute();
```

9. Complete Workflow Traces

9.1 Password login

1. User enters email and password in LoginFrame.
2. passwordLogin() reads the fields.
3. AuthService.authenticate() validates non-null input and format.
4. ValidationUtil checks the institutional email and password length.
5. UserDAO.findByEmail() executes a prepared query.
6. PasswordUtil.verifyPassword() checks BCrypt.
7. SessionContext.setCurrentUser(user) stores the session.
8. LoginFrame.openDashboard() checks the role.
9. AdminDashboardFrame opens for admin; StudentDashboardFrame opens for student.

9.2 Google login

1. User clicks Google login.
2. LoginFrame starts SwingWorker.
3. AuthService.loginWithGoogle() calls GoogleOAuthService.
4. OAuth service starts a local callback server.
5. It creates state and PKCE values.
6. It opens Google in the browser.
7. Google redirects to /callback.
8. The service checks the callback state.
9. It exchanges the authorization code.
10. It retrieves the profile.
11. AuthService validates @tkmce.ac.in.
12. It finds or creates a student account.
13. It stores the session and returns the user.
14. The UI opens the student dashboard.

9.3 Student submission

1. Student opens StudentDashboardFrame.
2. Student clicks New Submission.
3. SubmissionFormPanel collects the fields.
4. The form creates a Submission object.
5. The callback calls SubmissionService.createSubmission(...).
6. DatabaseSubmissionService delegates to SubmissionDAO.createWithReceipt(...).
7. DAO inserts the anonymous submissions row.
8. DAO reads the generated ID.
9. DAO inserts (student_id, submission_id) into the receipt table.
10. DAO commits the transaction.
11. Dashboard reloads IDs through the receipt path.
12. Student sees the ticket in their own table.

9.4 Student hype

1. Student selects a ticket.
2. Dashboard reads its submission ID.
3. HypeService.addHype() is called.
4. SubmissionHypeDAO checks or inserts the vote.
5. The database unique constraint prevents a duplicate pair.
6. Dashboard reloads the list and displays the count.

7. SubmissionDAO.findAll() orders submissions using hype count and creation time.

9.5 Admin resolution

1. Admin logs in.
2. AdminDashboardFrame verifies SessionContext.isAdmin().
3. The dashboard calls listAllSubmissions().
4. The service retrieves the queue through SubmissionDAO.
5. Admin selects a row.
6. Admin chooses a new status.
7. Service reads the current submission.
8. DAO updates the status.
9. DAO inserts a submission_history record.
10. Both operations commit in one transaction.
11. Admin may add a ResolutionNote.
12. Student can read the resulting history/note through the service flow.

10. Testing And Verification

10.1 Current test classes

Test class	What it checks
AuthServiceTest	Authentication validation and authentication behavior
DatabaseSubmissionServiceTest	Submission service operations
HypeServiceTest	Hype behavior
PasswordUtilTest	BCrypt hashing and verification
ValidationUtilTest	TKMCE email and password rules
AdminAnonymityRegressionTest	Admin path does not expose student identity

The current Maven run reports 12 tests passing.

10.2 How to study a test

For every test, identify:

1. What input is given?
2. What class is being tested?
3. What result is expected?
4. Which rule does the test protect?
5. What future code change could break it?

10.3 Manual golden path

When database configuration is available, verify:

1. Start with seeded data.
2. Log in as a student.
3. Create an issue or suggestion.
4. Confirm it appears in the student’s own list.
5. Add hype to a ticket.
6. Log out.
7. Log in as an admin.
8. Confirm the admin queue shows submissions without author identity.
9. Change a status.
10. Add a resolution note.
11. View history.
12. Log out.
13. Log in as the student again.
14. Confirm the student can see the update for their own ticket.

10.4 Verification evidence

For the rubric, keep evidence such as:

- Source files with the member’s name or ownership mapping.
- SQL schema and seed data.
- Test output.
- Screenshots of working student/admin flows.
- A written explanation of a technical decision.
- A failed case and the fix that resolved it.
- A planned-versus-completed table.

11. Five Member Learning Tracks

The official ownership matrix is recorded in docs/ARCHITECTURE.md and the first presentation. The sections below follow that matrix. The current branch may not contain every class named by the approved design, so each track separates implemented files from planned/documented work.

Part A ? Mohammed Nafih

Assigned responsibility

RBAC, Anonymity Vault, and Security Pipeline

Main files to study

- src/main/java/aura/service/AuthService.java
- src/main/java/aura/service/GoogleOAuthService.java
- src/main/java/aura/dao/StudentReceiptDAO.java
- src/main/java/aura/util/PasswordUtil.java
- src/main/java/aura/util/ValidationUtil.java
- src/main/java/aura/util/SessionContext.java
- src/test/java/aura/service/AuthServiceTest.java
- src/test/java/aura/util/PasswordUtilTest.java
- src/test/java/aura/util/ValidationUtilTest.java
- src/test/java/aura/security/AdminAnonymityRegressionTest.java
- docs/SECURITY.md

What to learn first

Learn Java method calls, constructors, try/catch, SQLException, static utility methods, regular expressions, password hashing, and session state.

What this part implements

1. Validate an official email address.
2. Validate the password length.
3. Find a user through UserDAO.
4. Verify a BCrypt hash.
5. Store the authenticated user in SessionContext.
6. Restrict admin and student screens by role.
7. Perform Google OAuth with state and PKCE.
8. Keep student ownership in the receipt vault.

Security walkthrough

- ValidationUtil rejects non-TKMCE email addresses.
- PasswordUtil.hashPassword() prevents plaintext persistence.
- PasswordUtil.verifyPassword() checks a candidate password.
- SessionContext stores the current user.
- Dashboard constructors enforce role checks.
- StudentReceiptDAO accesses the private receipt table.
- AdminAnonymityRegressionTest protects the no-author-exposure rule.

Java verification examples

```
public boolean isCurrentUser(User user) {
    User currentUser = SessionContext.getCurrentUser();
    return user != null
            && currentUser != null
            && currentUser.getUserId() == user.getUserId();
}
```

Explain every condition:

- user != null: there must be a user to compare.
- currentUser != null: a session must exist.
- IDs must match: the object must belong to the current session.

Demonstration to prepare

1. Show an invalid external email being rejected.
2. Show a valid TKMCE email being normalized.
3. Explain where the BCrypt hash is created.
4. Show how a successful login sets SessionContext.
5. Show why an admin cannot open the student frame.
6. Show why submissions has no student_id.
7. Show how the receipt lookup finds a student's own submission.

Remaining learning work

- Understand OAuth configuration rather than only mock login.
- Verify missing configuration errors.
- Test unauthorized paths and receipt ownership edge cases.
- Explain the security design without claiming that mock mode is production authentication.

Part B ? Muhammed Rinshid

Assigned responsibility

Team Lead: System Coordination, Core Architecture, and Supabase Integration

Main files to study

- src/main/java/aura/Main.java
- src/main/java/aura/config/DatabaseConfig.java
- src/main/java/aura/config/OAuthConfig.java
- src/main/java/aura/ui/LoginFrame.java
- pom.xml
- src/main/resources/db.properties.example
- src/main/resources/oauth.properties.example
- README.md
- docs/ARCHITECTURE.md
- docs/PRD.md
- docs/TRD.md

What to learn first

Learn packages, Maven, application startup, configuration separation, dependency direction, Swing event-thread startup, JDBC connection creation, and integration testing.

What this part coordinates

1. Start the application.
2. Apply FlatLaf.
3. Open the login screen.
4. Load database configuration.
5. Connect Java to Supabase/PostgreSQL.
6. Keep secrets outside committed source.
7. Coordinate the boundaries between member modules.

Startup walkthrough

Main.main():

1. Attempts to set FlatDarkLaf.
2. Calls SwingUtilities.invokeLater.
3. Creates LoginFrame.
4. Makes the frame visible.

Database configuration walkthrough

DatabaseConfig.getConnection():

1. Reads db.url.
2. Reads db.user.
3. Reads db.password.
4. Loads the configured driver.
5. Calls DriverManager.getConnection.
6. Returns a connection to the DAO.

The UI does not know the password or connection details. This is configuration separation.

Integration responsibility

The team lead should know the complete request path:

```
Main
  -> LoginFrame
    -> AuthService
      -> UserDAO / DatabaseSubmissionService
        -> DatabaseConfig
          -> PostgreSQL/Supabase
```

Java verification examples

```
SwingUtilities.invokeLater(() -> {
    LoginFrame loginFrame = new LoginFrame();
    loginFrame.setVisible(true);
});
```

Explain why the UI is created inside invokeLater and why Main does not contain database SQL.

Demonstration to prepare

1. Run the application entry point.
2. Explain the Maven dependencies in pom.xml.
3. Show how the local properties files avoid hard-coded credentials.
4. Explain how a DAO receives a configured connection.
5. Draw the complete architecture from UI to database.
6. Explain how member contracts allow five people to work independently.

Remaining learning work

- Verify the Supabase connection with real local configuration.
- Run the end-to-end golden path.
- Coordinate merging, test failures, and configuration setup.
- Document any environment-specific issues without placing secrets in Git.

Part C ? Nirmal

Assigned responsibility

Proposed Solution, Models, UML, and Hype/Trending Engine

Main files to study

- src/main/java/aura/model/User.java
- src/main/java/aura/model/Submission.java
- src/main/java/aura/model/ResolutionNote.java
- src/main/java/aura/model/SubmissionHistory.java
- src/main/java/aura/dao/SubmissionHypeDAO.java
- src/main/java/aura/service/HypeService.java
- src/test/java/aura/service/HypeServiceTest.java
- docs/ARCHITECTURE.md
- docs/PRD.md
- docs/DATABASE.md

What to learn first

Learn Java classes, fields, constructors, getters/setters, object relationships, List, DAO delegation, unique database constraints, and sorting/aggregation.

Model responsibility

A model answers: ?What data does this part of the domain carry??

- User carries identity and role data.
- Submission carries anonymous ticket data.
- ResolutionNote carries administrator response data.
- SubmissionHistory carries state transition data.

The current branch uses strings for role, type, priority, and status. The documentation describes a future or approved enum design, but the actual current source must be studied first.

Hype responsibility

A hype is a student support vote. The database prevents the same student from hyping the same submission twice:

```
CONSTRAINT uq_hype_once_per_student
UNIQUE (submission_id, student_id)
```

HypeService is the business-facing class. SubmissionHypeDAO is the database-facing class.

Hype workflow

1. Dashboard reads the selected submission ID.
2. Dashboard calls HypeService.addHype.
3. Service calls SubmissionHypeDAO.
4. DAO inserts the student/submission pair.
5. The database rejects duplicates.
6. SubmissionDAO.findAll() counts hypes and sorts the queue.

Java verification examples

```
public int getHypeCount(int submissionId) {
    try {
        return hypeDAO.countForSubmission(submissionId);
    } catch (SQLException exception) {
        throw new IllegalStateException("Unable to count hypes.", exception);
    }
}
```

Explain why the service catches SQLException and exposes an application-level failure rather than making the UI understand database internals.

Demonstration to prepare

1. Explain each field in Submission.
2. Draw relationships among User, Submission, ResolutionNote, and SubmissionHistory.
3. Add a hype vote.
4. Try the same vote again and explain the uniqueness rule.
5. Show how hype count affects list ordering.
6. Explain why a model does not open a database connection.

Easier scoring focus

This is one of the more structured parts to defend because the relationships are visible in the model classes, SQL tables, service methods, and hype test. Prepare a clear class-to-table-to-workflow explanation and use the real method names.

Remaining learning work

- Align the documented UML and current model source carefully.
- Add or strengthen tests for duplicate hype and count ordering.
- Understand whether typed enums are required for the final approved implementation before changing public APIs.

Part D ? Rahandeep

Assigned responsibility

Database Engineering and FlatLaf Desktop GUI

Main files to study

- sql/schema.sql
- sql/seed.sql
- src/main/java/aura/dao/UserDAO.java
- src/main/java/aura/dao/SubmissionDAO.java
- src/main/java/aura/ui/student/StudentDashboardFrame.java
- src/main/java/aura/ui/student/SubmissionFormPanel.java
- src/main/java/aura/ui/LoginFrame.java
- src/test/java/aura/service/DatabaseSubmissionServiceTest.java
- docs/DATABASE.md
- docs/SECURITY.md

What to learn first

Learn relational tables, primary and foreign keys, SQL INSERT/SELECT/UPDATE/DELETE, JDBC resource handling, Swing components, listeners, table models, and transactions.

Database responsibility

You should be able to explain:

- Why users is created before dependent tables.
- Why submissions intentionally has no student ID.
- Why receipts need foreign keys.
- Why hype needs a unique pair.
- Why status/history updates use a transaction.
- Why indexes improve queue and receipt lookups.

DAO responsibility

UserDAO maps users.

SubmissionDAO is the largest current DAO. It:

- Saves submissions.
- Finds one submission.
- Finds all submissions.
- Creates a submission and receipt.
- Updates status.
- Records history.
- Adds resolution notes.
- Reads notes and history.
- Deletes submissions.

GUI responsibility

StudentDashboardFrame:

- Verifies the student session.
- Loads submissions.

- Displays a non-editable JTable.
- Opens the form dialog.
- Adds hype.
- Shows details.
- Logs out.

SubmissionFormPanel collects the new ticket fields.

Java/JDBC verification example

```
try (Connection connection = DatabaseConfig.getConnection();
     PreparedStatement statement = connection.prepareStatement(sql,
             Statement.RETURN_GENERATED_KEYS)) {
    bindSubmission(statement, submission);
    statement.executeUpdate();
    try (ResultSet keys = statement.getGeneratedKeys()) {
        if (keys.next()) {
            submission.setSubmissionId(keys.getInt(1));
        }
    }
}
```

Explain:

- try-with-resources closes objects.
- PreparedStatement binds values safely.
- Generated keys return the new submission ID.
- The ID is required for the receipt.

Demonstration to prepare

1. Run the schema in order.
2. Explain every table and key.
3. Show one DAO SELECT and one DAO INSERT.
4. Explain the generated-key flow.
5. Open the student dashboard.
6. Create a ticket through the form.
7. Show the table refresh.
8. Explain how the Java listener calls a service.

Easier scoring focus

This is a highly demonstrable section because the schema, SQL, JDBC code, and student screen can be shown directly. Prepare diagrams of one complete operation from button click to database row.

Remaining learning work

- Expand DAO integration tests.
- Test missing connection configuration.
- Improve form validation and UI error/loading states.
- Confirm the database schema and current Java mappings remain synchronized.

Part E ? Adil Rahman

Assigned responsibility

Audit Lifecycle, Admin Dashboard, and Analytics

Main files to study

- src/main/java/aura/ui/admin/AdminDashboardFrame.java
- src/main/java/aura/service/DatabaseSubmissionService.java
- src/main/java/aura/service/SubmissionService.java
- src/main/java/aura/model/SubmissionHistory.java
- src/main/java/aura/model/ResolutionNote.java
- sql/schema.sql
- sql/seed.sql
- docs/ROADMAP.md
- docs/TESTING.md

Planned/documented classes to identify honestly

Some design documents mention TrackingService, ReportService, and ResolutionNoteDAO. They are part of the approved/documented target architecture, but they are not present as separate Java files in the current source tree. The current implementation places relevant operations in:

- DatabaseSubmissionService.
- SubmissionService.
- SubmissionDAO.
- AdminDashboardFrame.

This distinction is important in a technical explanation: describe what exists, then explain what still needs to be extracted or completed.

Admin workflow responsibility

AdminDashboardFrame:

1. Checks for an authenticated admin.
2. Loads all submissions.
3. Displays ID, title, status, and hype.
4. Lets the admin change status.
5. Lets the admin add a resolution note.
6. Shows history and notes.
7. Refreshes the queue.
8. Logs out.

Audit lifecycle

The current lifecycle values are:

```
PENDING
  -> ASSIGNED
    -> IN_PROGRESS
      -> RESOLVED

PENDING or other valid state
  -> REJECTED
```

The database records both the current state and the history of transitions.

Status update code idea

The DAO operation performs two writes in one transaction:

```
UPDATE submissions
SET status = ?
WHERE submission_id = ?;

INSERT INTO submission_history
(submission_id, old_status, new_status, changed_by)
VALUES (?, ?, ?, ?);
```

If either write fails, the transaction rolls back.

Analytics and reports status

The approved requirements describe dashboard metrics and CSV reports. The current branch has the queue, hype count, history, and notes, but does not contain separate ReportService or TrackingService classes. These are remaining implementation work rather than completed source modules.

Demonstration to prepare

1. Log in as an admin.
2. Explain the role check.
3. Show the queue and the fields displayed.
4. Change a ticket status.
5. Explain how the old and new status are stored.
6. Add a resolution note.
7. View history.
8. State clearly that report generation and analytics require the next implementation phase.

Easier scoring focus

This section has a visible workflow and a strong database audit story. To score well, connect the UI control, service method, DAO transaction, submission_history row, and displayed result. Do not claim that analytics/report classes exist until they are implemented.

Remaining learning work

- Implement a dedicated report output path.
- Decide and implement analytics metrics.
- Add richer filters and dashboard summaries.
- Add tests for invalid state transitions and rollback behavior.
- Decide how notifications should be represented in the final scope.

12. Planned Versus Completed Work

This table is important for both learning and the rubric. It prevents the team from confusing a design document with an implemented class.

Area	Planned/documented	Present in current branch	Current understanding
Requirements	PRD/TRD and use cases	Yes	Completed documentation foundation
Architecture	Four-layer flow and diagrams	Yes	Implemented as the intended direction
Database	Six-table schema and seed data	Yes	Core persistence implemented
Models	User/submission/note/history	Yes	Current model classes exist
Enums	Role/type/category/priority/status package	Not as a current package	Current code uses strings; align before finalizing
User DAO	User CRUD	Yes	Implemented
Submission DAO	Submission, receipt, history, notes	Yes	Implemented in SubmissionDAO
Receipt DAO	Private receipt lookup	Yes	Implemented
Hype DAO/service	Vote and count	Yes	Implemented
Authentication	Password, OAuth structure, session	Yes	Core implemented; real OAuth needs configuration
Student UI	Login, dashboard, form	Yes	Core workflow implemented; polish remains
Admin UI	Queue, status, notes, history	Yes	Core workflow implemented
TrackingService	Separate service in design	No separate class	Current operations are in service/DAO flow
ReportService	Reports in design	No separate class	Pending
Analytics	Metrics in requirements/design	Partial design only	Pending implementation
Notifications	Requirements/design reference	No complete implementation	Pending
Tests	JUnit and anonymity regression	Yes, 12 current passing tests	More DAO/integration coverage needed
Final UI polish	Planned	Partial	Pending

13. Rubric Evidence Checklist

The supplied rubric image asks for four areas. Use this checklist when preparing evidence.

13.1 Significant progress beyond Phase 1

Show:

- Working Java source beyond the original design.
- SQL schema and seed data.
- Login and session behavior.
- Student and admin screens.
- Submission, hype, status, note, and history workflows.
- Test output showing the current baseline.

13.2 Technologies and programming concepts

Be ready to explain:

- Java classes and objects.
- Interfaces such as SubmissionService.
- Encapsulation in model fields.
- Collections such as List<Submission>.
- Exceptions and SQLException handling.
- Swing listeners and event-driven UI.
- Maven dependencies.
- JDBC and PreparedStatement.
- PostgreSQL constraints and indexes.
- BCrypt.
- OAuth state and PKCE.
- Transactions.
- Role checks and privacy boundaries.

13.3 Individual contribution

Each member must be able to show:

1. The assigned package/files.
2. A real method they can explain.
3. A requirement supported by that code.
4. A technical decision.
5. A challenge or limitation.
6. A test or demonstration.
7. The next task still needed.

13.4 Planned versus completed work

Every explanation should use accurate language:

- ?Implemented in the current branch? when the class and behavior exist.
- ?Partially implemented? when the core exists but polish/tests are incomplete.
- ?Documented/planned? when the design mentions it but the class is absent.
- ?Pending? when it belongs to the remaining 30%.

14. Final Learning And Build Plan

Phase A: Understand

- Read this manual.
- Open every file in your member section.
- Draw the call chain for one workflow.
- Explain one table and one Java model.

Phase B: Reproduce

- Configure Java and Maven.
- Run mvn test.
- Apply schema.sql and seed.sql.
- Configure db.properties.
- Start the application.
- Complete the golden path.

Phase C: Verify

- Test invalid email.
- Test invalid password.
- Test unauthorized dashboard access.
- Test duplicate hype.
- Test missing database configuration.
- Test status history.
- Confirm no student ID appears in public submission data.

Phase D: Teach Each Other

Each member should teach another member:

- What their classes do.
- What calls them.
- What data they receive and return.
- What database tables they use.
- What can fail.
- What test proves the behavior.

Phase E: Complete The Remaining 30%

1. Implement reporting.
2. Implement analytics.
3. Decide and implement notifications.
4. Improve history and state validation.
5. Polish both dashboards and the form.
6. Add wider DAO/integration tests.
7. Fix discovered bugs.
8. Update documentation to match source.
9. Run the complete test suite.
10. Perform the final golden-path walkthrough.

15. Final Technical Questions

Every member should eventually answer these without reading the manual:

1. What problem does AURA solve?
2. Why is AURA a Java desktop application?
3. Why does the project use Swing and FlatLaf?
4. What does Maven do?
5. What is the difference between a UI class, service, DAO, and model?
6. Where does SQL execute?
7. How does Java connect to PostgreSQL?
8. Why is PreparedStatement used?
9. Why is student_id missing from submissions?
10. How does the receipt vault work?
11. How does password login work?
12. How does Google OAuth work in this project?
13. What does SessionContext store?
14. How does an admin change a status?
15. Why are status update and history insertion in one transaction?
16. How does hype prevent duplicate votes?
17. Which current classes are planned in documentation but absent from code?
18. What is completed now?
19. What remains in the 30%?
20. Which member owns each technical area?

If a member can answer these questions, trace the source files, run the tests, and demonstrate one feature, the team has moved from merely possessing an agent-generated project to understanding and owning the project technically.

Source Files Used For This Manual

- AGENT.md
- README.md
- pom.xml
- docs/AURA.pdf
- docs/ARCHITECTURE.md
- docs/AUTH_TASK_GUIDE.md
- docs/DATABASE.md
- docs/PRD.md
- docs/ROADMAP.md
- docs/SECURITY.md
- docs/TESTING.md
- docs/TRD.md
- sql/schema.sql
- sql/seed.sql
- All current files under src/main/java/aura/
- All current files under src/test/java/aura/
- The supplied rubric image: needed to include.jpeg